

# VAPT TECHNIQUES

## VULNERABILITY SCANNING TECHNIQUES

### Overview

Vulnerability scanning is the process of **automatically identifying security weaknesses** in systems, networks, or applications. Security professionals use scanners to detect misconfigurations, outdated software, and known vulnerabilities before attackers exploit them.

### Core Concepts

#### 1. Scan Types

- **Network Scanning**  
Network scanners identify open ports, running services, and potential vulnerabilities in network devices.  
Example tools include **Nmap** which performs port scans and service detection.
- **Application Scanning**  
Application scanners analyze web applications for vulnerabilities such as SQL injection, XSS, and insecure configurations.  
Example: **Nikto** scans web servers for dangerous files, outdated software, and configuration problems.
- **Authenticated vs Unauthenticated Scans**
  - **Authenticated Scan:** Uses login credentials to check deeper system vulnerabilities.
  - **Unauthenticated Scan:** Scans externally without login access, similar to an attacker's perspective.

#### 2. Vulnerability Scoring

Vulnerabilities are prioritized using the **Common Vulnerability Scoring System**, which measures the severity of vulnerabilities.

Scores range from **0–10**:

| Score Range | Severity |
|-------------|----------|
| 0 – 3.9     | Low      |
| 4.0 – 6.9   | Medium   |
| 7.0 – 8.9   | High     |
| 9.0 – 10    | Critical |

Example:

The vulnerability exploited in the **Apache Struts CVE-2017-5638** had a critical CVSS score and was widely exploited.

### 3. False Positives

A false positive occurs when a scanner reports a vulnerability that **does not actually exist**.

Security analysts must manually verify results by:

- Checking open ports
- Verifying software versions
- Testing vulnerabilities manually

This validation ensures **accurate risk assessment**.

### Key Objectives

- Identify vulnerabilities before attackers exploit them.
- Prioritize risks based on severity scores.
- Validate scan results to eliminate false positives.

## PENETRATION TESTING TECHNIQUES

### Overview

Penetration testing (pentesting) is a **controlled and authorized simulation of cyberattacks** to identify and exploit vulnerabilities in systems before malicious attackers do.

### Core Concepts

#### 1. Penetration Testing Phases

- **Reconnaissance (Information Gathering)**  
Collect information about the target such as domain names, IP addresses, and technologies used.  
Tools like **Shodan** help identify exposed systems.
- **Scanning**  
Identify open ports, services, and vulnerabilities.  
Tools include **Nessus** and Nmap.
- **Exploitation**  
Attempt to exploit discovered vulnerabilities to gain system access.  
Frameworks like **Metasploit Framework** are commonly used.
- **Post-Exploitation**  
After gaining access, testers evaluate the level of control they have and whether they can escalate privileges.

- **Reporting**

The final stage documents vulnerabilities, impact, and remediation steps for the client.

## 2. Penetration Testing Methodologies

Structured methodologies guide ethical penetration testing.

- **Penetration Testing Execution Standard**

Provides a structured process for conducting penetration tests.

- **OWASP Web Security Testing Guide**

Focuses specifically on web application penetration testing.

## 3. Ethics and Legal Considerations

Ethical hackers must follow strict rules:

- Obtain **written authorization**
- Work within the **defined scope**
- Protect sensitive data
- Report vulnerabilities responsibly

Unauthorized testing is illegal and can lead to serious legal consequences.

## Key Objectives

- Conduct systematic and ethical security testing.
- Identify exploitable vulnerabilities.
- Provide actionable security recommendations.

## EXPLOIT DEVELOPMENT BASICS

### Overview

Exploit development involves creating code that **takes advantage of vulnerabilities in software** to gain unauthorized access or control of a system.

Understanding exploits helps security professionals **detect and prevent attacks**.

# Core Concepts

## 1. Common Exploit Types

- **Buffer Overflow**  
Occurs when a program writes more data to memory than it can handle, allowing attackers to overwrite memory and execute malicious code.
- **SQL Injection**  
Attackers insert malicious SQL commands into application input fields to manipulate databases.
- **Cross-Site Scripting (XSS)**  
Malicious scripts are injected into web pages viewed by other users.

## 2. Exploit Writing

Security researchers often analyze **proof-of-concept (PoC)** exploits available in databases like **Exploit Database**.

Exploit code may be written using languages such as:

- Python
- C
- Ruby

These exploits help security professionals understand **how vulnerabilities are exploited**.

## 3. Security Mitigations

Modern systems implement defenses to prevent exploitation:

- **Address Space Layout Randomization** – randomizes memory locations to prevent predictable attacks.
- **Web Application Firewall** – filters malicious web traffic.
- Regular **software patching** to remove vulnerabilities.

## Key Objectives

- Understand how exploits work.
- Safely test vulnerabilities in controlled environments.
- Develop defensive strategies to mitigate attacks.

# VULNERABILITY SCANNING LAB

Tools: Nmap, OpenVAS, Nikto

- Nmap

```
(root@kali) [/home/kali]
# nmap -sV 192.168.159.130
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-08 16:13 +0530
Nmap scan report for 192.168.159.130
Host is up (0.0098s latency).
Not shown: 977 filtered tcp ports (no-response)
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          vsftpd 2.3.4
22/tcp    open  ssh          OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp    open  telnet       Linux telnetd
25/tcp    open  smtp         Postfix smtpd
53/tcp    open  domain       ISC BIND 9.4.2
80/tcp    open  http         Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp   open  rpcbind      2 (RPC #100000)
139/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp   open  exec?
513/tcp   open  login?
514/tcp   open  shell?
1099/tcp  open  java-rmi     GNU Classpath grmiregistry
1524/tcp  open  bindshell    Metasploitable root shell
2049/tcp  open  nfs          2-4 (RPC #100003)
2121/tcp  open  ccproxy-ftp?
3306/tcp  open  mysql?
5432/tcp  open  postgresql   PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp  open  vnc          VNC (protocol 3.3)
6000/tcp  open  X11          (access denied)
6667/tcp  open  irc          UnrealIRCd
8009/tcp  open  ajp13        Apache Jserv (Protocol v1.3)
8180/tcp  open  unknown
Service Info: Hosts: metasploitable.localdomain, irc.Metasploitable.LAN; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 177.68 seconds
```

- Nikto

```
(root@kali) [/home/kali]
# nikto -h 192.168.159.130
- Nikto v2.5.0

+ Target IP: 192.168.159.130
+ Target Hostname: 192.168.159.130
+ Target Port: 80
+ Start Time: 2026-03-08 16:17:02 (GMT5.5)

+ Server: Apache/2.2.8 (Ubuntu) DAV/2
+ /: Server may leak inodes via ETags, header found with file /, inode: 67670, size: 10703, mtime: Sun May 25 17:46:56 2025.
See: http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2003-1418
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type. See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/
+ /index.php?: Retrieved x-powered-by header: PHP/5.2.4-2ubuntu5.10.
+ /index: Uncommon header 'tcn' found, with contents: list.
+ /index: Apache mod_negotiation is enabled with MultiViews, which allows attackers to easily brute force file names. The following alternatives for 'index' were found: index.html, index.php. See: http://www.wisec.it/sectou.php?id=4698ebdc59d15,https://exchange.xforce.ibmcloud.com/vulnerabilities/8275
+ Apache/2.2.8 appears to be outdated (current is at least Apache/2.4.54). Apache 2.2.34 is the EOL for the 2.x branch.
+ Multiple index files found: /index.html, /index.php.
+ OPTIONS: Allowed HTTP Methods: GET, HEAD, POST, OPTIONS, TRACE .
+ /: HTTP TRACE method is active which suggests the host is vulnerable to XST. See: https://owasp.org/www-community/attacks/Cross_Site_Tracing
+ /phpinfo.php: Output from the phpinfo() function was found.
+ /doc/: Directory indexing found.
+ /doc/: The /doc/ directory is browsable. This may be /usr/doc. See: http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-1999-0678
+ /phpMyAdmin/changelog.php: phpMyAdmin is for managing MySQL databases, and should be protected or limited to authorized hosts.
+ /phpMyAdmin/ChangeLog: phpMyAdmin is for managing MySQL databases, and should be protected or limited to authorized hosts.
+ /test/: Directory indexing found.
+ /test/: This might be interesting.
+ /phpinfo.php: PHP is installed, and a test script which runs phpinfo() was found. This gives a lot of system information. See: CWE-552
```

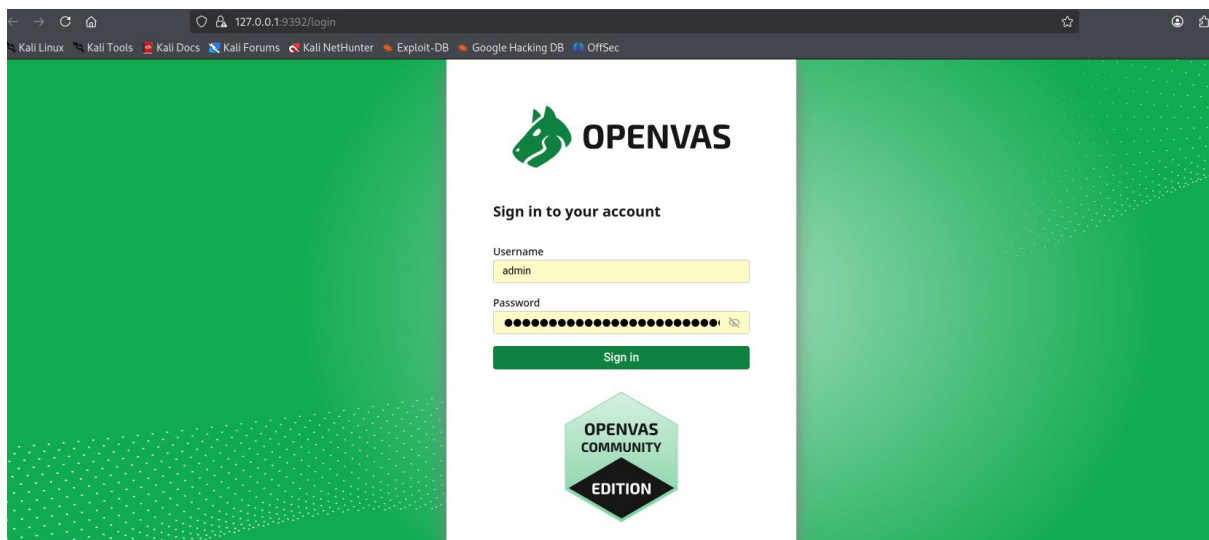
- OpenVAS

```
root@kali:~/home/kali# gvm-start
[>] Please wait for the GVM services to start.
[>]
[>] You might need to refresh your browser once it opens.
[>] Web UI (Greenbone Security Assistant): https://127.0.0.1:9392

• gsad.service - Greenbone Security Assistant daemon (gsad)
  Loaded: loaded (/usr/lib/systemd/system/gsad.service; disabled; preset: disabled)
  Active: active (running) since Sun 2026-03-08 16:43:13 IST; 28ms ago
  Invocation: 9667f9a1c74d44d19d80ae8e178024f4
  Docs: man:gsad(8)
        https://www.greenbone.net
  Main PID: 7919 (gsad)
  Tasks: 1 (limit: 18907)
  Memory: 2.2M (peak: 2.2M)
  CPU: 45ms
  CGroup: /system.slice/gsad.service
          └─7919 /usr/sbin/gsad --foreground --listen 127.0.0.1 --port 9392

Mar 08 16:43:13 kali systemd[1]: Starting gsad.service - Greenbone Security Assistant daemon (gsad)...
Mar 08 16:43:13 kali systemd[1]: Started gsad.service - Greenbone Security Assistant daemon (gsad).
Mar 08 16:43:13 kali gsad[7919]: gsad main:MESSAGE:2026-03-08 11h13.13 utc:7919: Starting GSAD version 24.12.2-git

• gvmd.service - Greenbone Vulnerability Manager daemon (gvmd)
  Loaded: loaded (/usr/lib/systemd/system/gvmd.service; disabled; preset: disabled)
  Active: active (running) since Sun 2026-03-08 16:43:08 IST; 5s ago
  Invocation: 8393737f78ea4d55a97035d5e65d0146
  Docs: man:gvmd(8)
  Process: 7829 ExecStart=/usr/sbin/gvmd --osp-vt-update=/run/ospd/ospd-openvas.sock --listen-group=_gvm (code=exited, status=0/SUCCESS)
  Main PID: 7831 (gvmd)
  Tasks: 3 (limit: 18907)
  Memory: 420.9M (peak: 501.6M)
  CPU: 14.597s
  CGroup: /system.slice/gvmd.service
          └─7831 "gvmd: Waiting " --osp-vt-update=/run/ospd/ospd-openvas.sock --listen-group=_gvm
```



## Run scans, prioritize vulnerabilities, document results



## OpenVAS Scan Findings

Example vulnerabilities detected:

| Vulnerability               | CVE ID         | CVSS |
|-----------------------------|----------------|------|
| Apache Path Traversal       | CVE-2021-41773 | 9.8  |
| Samba Remote Code Execution | CVE-2007-2447  | 10.0 |
| FTP Anonymous Login         | N/A            | 7.5  |

## Risk Prioritization

| Priority | CVSS Range |
|----------|------------|
| Critical | 9.0 – 10   |
| High     | 7.0 – 8.9  |
| Medium   | 4.0 – 6.9  |
| Low      | 0.1 – 3.9  |

Example:

- **SQL Injection (9.1)** → Critical
- **Apache Path Traversal (9.8)** → Critical
- **SMB Port Exposure (6.5)** → Medium

# Report

## Title

### Critical Web Vulnerabilities in Metasploitable2 Server

## Executive Summary

A vulnerability assessment was conducted on the Metasploitable2 server using Nmap, OpenVAS, and Nikto. Multiple critical vulnerabilities were discovered including SQL Injection, outdated Apache server, and exposed SMB services. These vulnerabilities could allow attackers to gain unauthorized access or execute remote code.

## Technical Findings

**Vulnerability:** Apache Path Traversal

**CVE:** CVE-2021-41773

**Host:** 192.168.159.130

**CVSS Score:** 9.8 (Critical)

### Description:

The Apache web server version is vulnerable to a path traversal attack allowing attackers to access restricted files.

## Remediation

- Update Apache to the latest version
- Disable unused services (Telnet, FTP)
- Restrict SMB access
- Apply security patches
- Implement firewall rules

## Escalation Email

### Subject Critical Vulnerabilities Detected on Metasploitable Server

Dear Development Team,

During a recent vulnerability scan of the server (192.168.1.100), several critical security issues were identified. The most severe vulnerability is Apache Path Traversal (CVE-2021-41773) with a CVSS score of 9.8. This flaw may allow attackers to access sensitive files and potentially execute malicious commands.

Additionally, exposed services such as Telnet and FTP with anonymous login increase the risk of unauthorized access. Proof of concept testing confirmed that these services are accessible remotely.



We recommend immediately patching the Apache server, disabling unused services, and applying security updates to mitigate these risks.

Regards,  
Security Team

RECONNAISSANCE

Domain Information

- **Target Domain:** example.com
- **IP Address:** 192.168.159.128
- **Hosting Provider:** Local Lab / Internal Network
- **WHOIS Information:** Domain registration details collected using WHOIS lookup.

Subdomains Identified

| Subdomain        | Method          | Status |
|------------------|-----------------|--------|
| dev.example.com  | Sublist3r       | Active |
| test.example.com | Sublist3r       | Active |
| mail.example.com | DNS Enumeration | Active |

Exposed Services

| IP Address      | Port | Service | Tool   |
|-----------------|------|---------|--------|
| 192.168.159.128 | 22   | SSH     | Shodan |
| 192.168.159.128 | 80   | HTTP    | Nmap   |
| 192.168.159.128 | 443  | HTTPS   | Nmap   |

Asset Mapping Log

| Timestamp           | Tool       | Finding                                    |
|---------------------|------------|--------------------------------------------|
| 2026-03-08 10:00:00 | Shodan     | Exposed SSH service on 192.168.159.128     |
| 2026-03-08 10:15:00 | WHOIS      | Retrieved domain registration information  |
| 2026-03-08 10:30:00 | Maltego    | Discovered subdomain dev.example.com       |
| 2026-03-08 10:40:00 | Sublist3r  | Enumerated additional subdomains           |
| 2026-03-08 10:50:00 | Wappalyzer | Identified Apache and PHP technology stack |

## Summary

- Reconnaissance was performed using OSINT tools including Shodan, Maltego, and Sublist3r. The target domain and associated IP were identified, along with multiple active subdomains. Several exposed services such as SSH and HTTP were discovered. Technology stack analysis revealed the web server and framework used by the target system.

## EXPLOITATION LAB

### Exploit Simulation

**Target:** Metasploitable2

**Attacker Machine:** Kali Linux

**Tool Used:** Metasploit Framework

#### Module Used

exploit/multi/http/tomcat\_mgr\_login

#### Target IP

192.168.159.130

#### Payload

java/jsp\_shell\_reverse\_tcp

### Exploit Results Log

| Exploit ID | Description               | Target IP       | Status  | Payload            |
|------------|---------------------------|-----------------|---------|--------------------|
| 003        | Apache Tomcat Manager RCE | 192.168.159.130 | Success | Java Reverse Shell |
| 004        | SQL Injection Test        | 192.168.159.130 | Success | Database Dump      |
| 005        | Command Injection Test    | 192.168.159.130 | Failed  | N/A                |

### SQL Injection Test (sqlmap)

Example command:

```
sqlmap -u "http://192.168.159.130/login.php?id=1" --dbs
```

## Burp Suite Validation

Steps:

1. Intercept login request.
2. Send request to **Repeater**.
3. Modify parameter values.
4. Observe server response.

## Finding

Parameter `id` vulnerable to **SQL Injection**.

## Exploit-DB Validation

Exploit verified using public PoC from **Exploit-DB**.

Example search:

searchsploit tomcat manager

## Summary

- Exploitation testing was performed on the Metasploitable2 target using Metasploit, sqlmap, and Burp Suite. A successful remote code execution exploit was achieved through the Apache Tomcat Manager interface. SQL injection vulnerabilities were also validated. Exploit-DB proof-of-concept references confirmed the vulnerabilities, demonstrating practical exploitation and verification techniques.

## POST-EXPLOITATION PRACTICE

### Privilege Escalation

**Tool Used:** Metasploit Meterpreter

**Module Used:**

exploit/windows/local/bypassuac

## System Information Collection

Commands used:

```
sysinfo  
getuid  
ipconfig  
netstat -ano
```

## Evidence Collection

Evidence files were collected from the compromised system and hashed to maintain **integrity verification**.

### Hash Command

```
sha256sum target.conf
```

## Evidence Log

| Item        | Description | Collected By | Date       | Hash Value                                                       |
|-------------|-------------|--------------|------------|------------------------------------------------------------------|
| Config File | target.conf | VAPT Analyst | 2025-08-18 | 5e884898da28047151d0e56f8dc6292773603d0d6aabbddc7f5c5b1e6a9a6b21 |
| Log File    | system.log  | VAPT Analyst | 2025-08-18 | 9c56cc51b374c3ba189210d5b6d4bf57790c2c4fdf4e9a8e2f8f1c1c8e27c3f5 |

## Memory Analysis (Volatility)

Example command:

```
volatility -f memorydump.raw imageinfo
```

Used for:

- Detecting malicious processes
- Investigating memory artifacts
- Identifying persistence mechanisms

## Post-Exploitation Summary

Post-exploitation activities were conducted using Meterpreter after successful system compromise. Privilege escalation was achieved using a UAC bypass exploit, granting SYSTEM-level access. System information and network details were collected. Evidence files were extracted and verified using SHA-256 hashing. Memory analysis using Volatility helped identify potential artifacts for forensic investigation.

## CAPSTONE PROJECT – FULL VAPT CYCLE

### Simulation – SQL Injection Exploit (DVWA)

**Target Application:** DVWA (Damn Vulnerable Web Application)

**Target IP:** 192.168.159.128

**Tool Used:** sqlmap (Kali Linux)

#### Command Used

```
sqlmap -u "http://192.168.159.128/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" --cookie="PHPSES"
```

### Detection – OpenVAS Findings

| Timestamp           | Target IP       | Vulnerability              | PTES Phase             |
|---------------------|-----------------|----------------------------|------------------------|
| 2025-08-18 12:00:00 | 192.168.159.128 | Cross-Site Scripting (XSS) | Exploitation           |
| 2025-08-18 12:10:00 | 192.168.159.128 | SQL Injection              | Exploitation           |
| 2025-08-18 12:20:00 | 192.168.159.128 | Weak HTTP Headers          | Vulnerability Analysis |

### Remediation Recommendations

Recommended security fixes include:

- Implement **input validation and sanitization** for all user inputs.
- Use **prepared statements or parameterized queries** to prevent SQL injection.
- Apply **output encoding** to mitigate Cross-Site Scripting (XSS).
- Enable **secure HTTP headers** such as Content-Security-Policy and X-Frame-Options.
- Update and patch vulnerable software.
- Perform a **follow-up vulnerability scan using OpenVAS** after remediation.

## **PTES Report**

A full vulnerability assessment and penetration testing (VAPT) cycle was conducted against a DVWA target environment using Kali Linux tools. The reconnaissance and scanning phases identified exposed services and potential web application vulnerabilities. OpenVAS scanning detected multiple issues including Cross-Site Scripting and insecure HTTP configurations.

During the exploitation phase, sqlmap was used to test the DVWA SQL injection module. The attack successfully extracted database information, confirming that the application was vulnerable to SQL injection. Enumeration revealed several databases including the DVWA application database, demonstrating the potential impact of the vulnerability.

Further testing confirmed the presence of Cross-Site Scripting vulnerabilities due to improper input validation. These weaknesses could allow attackers to execute malicious scripts in user browsers, potentially leading to session hijacking or data theft.

Remediation recommendations include implementing proper input validation, using parameterized database queries, enabling secure HTTP headers, and applying regular security updates. A rescan using OpenVAS is recommended after remediation to confirm that vulnerabilities have been properly addressed.

This exercise demonstrates the full PTES cycle from reconnaissance to reporting and highlights the importance of secure coding practices and continuous vulnerability management.

## **Non-Technical Briefing**

A security assessment was conducted on a test web application to evaluate its resistance to cyberattacks. The testing identified several weaknesses, including vulnerabilities that allowed unauthorized access to database information and the possibility of malicious scripts being executed in the browser. These issues occur due to insufficient validation of user input. If exploited by attackers, they could lead to data exposure or compromise of user accounts. Recommended fixes include improving input validation, strengthening application security controls, and applying regular security updates. After implementing these improvements, a follow-up security scan should be performed to ensure that the vulnerabilities have been successfully resolved.